Coding & Solution

Date	July 9, 2026
Team ID	SWTID-2026-1262
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution Summary

Field	Details
Repository Link / URL	https://github.com/HariCharan-1912/OptiCrop-Smart-Agricultural-Production-Optimization-Engine.git
Programming Language(s)	Python 3.10+ (Tested successfully on Python 3.13 via Anaconda base environment)
Framework(s) Used	Flask Micro Web Framework
Key Features Implemented	Automated Multi-Model Evaluator (train_model.py): Trains and compares Logistic Regression, KNN, Decision Tree, and Random Forest algorithms over a synthetic 1,500-sample matrix, explicitly bypassing Pylance type-checking exceptions.Model Serialization Pipeline: Successfully compiles and persists the top-performing Random Forest model structure into a local model.pkl binary store.Multi-Scenario Routing Framework (app.py): Drives distinct data flows for Scenario 1 (Crop Recommendation via ML Inference), Scenario 2 (Target Suitability and Mismatch Audit), and Scenario 3 (Macro Policy & Efficiency Planning Framework).Agro-Ecological Field Sanitizer: Implements strict backend range-boundary checks handling floats for N, P, K, temperature, humidity, ph, and rainfall.
Pending / Incomplete Features	None. All primary functional objectives mapped out across the three core operational scenarios have been successfully implemented and tested locally.
Setup / Run Instructions	1. Open the project root directory inside **VS Code**. 2. Activate your conda base environment in the terminal terminal via: C:\Users\haric\anaconda3\Scripts\activate 3. Install the environment packages: python -m pip install -r requirements.txt 4. Run the machine learning training pipeline to export the binary: python train_model.py 5. Boot up the local web deployment server loop: python app.py 6. Access the running dashboard interface via your browser at http://127.0.0.1:5000/.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The application successfully runs using pre-compiled native Anaconda distribution packages, completely avoiding local platform compilation or heavy C++ toolchain errors on Windows machines. The modular integration between the Scikit-Learn tree inference matrices and the Flask web layers provides a responsive, zero-code environment optimal for traditional agricultural workflows.